

167. 两数之和 II - 输入有序数组

给你一个下标从 1 开始的整数数组 `numbers`，该数组已按 **非递减顺序排列**，请你从数组中找出满足相加之和等于目标数 `target` 的两个数。如果设这两个数分别是 `numbers[index1]` 和 `numbers[index2]`，则：

```
1 <= index1 < index2 <= numbers.length
```

以长度为 2 的整数数组 `[index1, index2]` 的形式返回这两个整数的下标 `index1` 和 `index2`。

你可以假设每个输入 **只对应唯一的答案**，而且你 **不可以重复使用相同的元素**。

你所设计的解决方案必须只使用 **常量级的额外空间**。

示例 1

```
输入: numbers = [2,7,11,15], target = 9
输出: [1,2]
解释: 2 与 7 之和等于目标数 9 。因此 index1 = 1, index2 = 2 。
```

示例 2

```
输入: numbers = [2,3,4], target = 6
输出: [1,3]
解释: 2 与 4 之和等于目标数 6 。因此 index1 = 1, index2 = 3 。
```

示例 3

```
输入: numbers = [-1,0], target = -1
输出: [1,2]
解释: -1 与 0 之和等于目标数 -1 。因此 index1 = 1, index2 = 2 。
```

提示

```
2 <= numbers.length <= 3 * 10^4
-1000 <= numbers[i] <= 1000
numbers 按 非递减顺序 排列
```

```
-1000 <= target <= 1000
```

仅存在一个有效答案

参考答案

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        vector<int> res(2);
        int l = 0;
        int r = numbers.size() - 1;

        while (l < r) {
            int temp = numbers[l] + numbers[r];

            if (temp > target) {
                r--;
            } else if (temp < target) {
                l++;
            } else if (temp == target) {
                res[0] = l + 1;
                res[1] = r + 1;
                break;
            }
        }

        return res;
    }
};
```

简单的总结：

利用了数组是有序的性质

如果数组 **不是有序**的

可以通过 **暴力枚举** 两次遍历 **for** 循环

如果数组 **是有序**的

就可以用 **双指针**

一个在数组的 **头部**

一个在数组的 **尾部**

如果两个数相加的和

是大于 **target** 的

大于这个目标值的

说明我需要把这两个数的和变小

也就是将 **右指针左移**
如果两个数相加的和小于 **target**
我就将 **左指针右移**
就这样不断地逼近答案

双指针的本质是缩小搜索范围

判断一个题能不能用双指针
就是判断能不能
通过移动双指针来 **安全地排除** 一些不需要的答案

一些信号

数组或者字符串是 **有序的**
两数之和
三数之和
最接近的三数之和
两数的平方和
接雨水
盛最多水的容器 这种
连续子数组
子串
滑动窗口
最长或最短满足条件的连续区间
这个时候双指针就不是往中间逼近了，而是 **滑动**
判断回文

双指针的思想是

移动其中一个指针
可以让某个值 **单调的变化**
从而 **排除一部分答案**

什么时候能用？

能不能用两个节点表示一个搜索范围
指针移动的时候结果会不会 **单调的变化**
指针移动的时候会不会 **安全地排除一部分答案**

指针类型

对撞双指针
滑动窗口